Proyecto 3
Algoritmo de Dijkstra

Josue gutierrez Andrade

Diseño y Análisis de Algoritmos

Resumen

Sobre la biblioteca de grafos de los proyectos anteriores se implementa el método `Dijkstra` de la clase `Grafo`. Dado un nodo fuente s, el método calcula el árbol de caminos más cortos y nombra cada nodo con su distancia a s; por ejemplo, `"2 (22.45)"`. El método se aplicó a 10 grafos con pesos aleatorios, generados con los cinco modelos aleatorios de la biblioteca con 30 y 500 nodos. Todos los grafos y árboles se exportaron a formato GraphViz y se visualizaron.

Índice

1. Introducción **2**

2. Pesos en la biblioteca de grafos **2**

3. Algoritmo de Dijkstra **2**

3.1. Descripción 2
3.2. Implementación 2
3.3. Complejidad y correctitud 3
3.4. Ejemplo 4
3.5. Verificación 4

4. Experimentos **4**

5. Resultados **5**

6. Visualización **6**

1. Introducción

En un grafo con pesos no negativos en las aristas, la longitud de un camino es la suma de los pesos de sus aristas. El problema del camino más corto consiste en encontrar, para cada nodo v, un camino de longitud mínima desde un nodo fuente s. Esa longitud mínima es la distancia $d(v)$. El algoritmo de Dijkstra resuelve el problema para todos los nodos a la vez. Si para cada nodo $v \neq s$ se guarda el nodo desde el que se llegó a él por el camino más corto, esas aristas forman un árbol con raíz en s: el *árbol de caminos más cortos*. En él, el único camino de s a v es un camino más corto en el grafo original.

2. Pesos en la biblioteca de grafos

Los grafos de los proyectos anteriores no tenían pesos, así que se hicieron estos cambios en la clase `Grafo`:

- `agregar_arista(u, v, peso)` guarda el peso en un diccionario `_pesos`, indexado por el par (u, v); en un grafo no dirigido, también por (v, u). Con él, `peso(u, v)` consulta el peso de una arista en tiempo $O(1)$.
- `asignar_pesos_aleatorios(minimo, maximo)` asigna a cada arista un peso uniforme en $[\text{minimo}, \text{maximo}]$, redondeado a dos decimales.
- `guardar_graphviz` escribe el peso en el atributo `weight`; por ejemplo, `"0" -- "26" [weight=11.07];`.

Los métodos `BFS`, `DFS_R` y `DFS_I` del Proyecto 2 se conservan sin cambios.

3. Algoritmo de Dijkstra

3.1. Descripción

Se sigue el algoritmo visto en clase. Sea S el conjunto de nodos explorados, para los cuales ya se conoce la distancia definitiva. Al inicio $S = \emptyset$, $d(s) = 0$ y la distancia tentativa de los demás nodos es ∞. En cada paso:

1. Se elige el nodo $u \notin S$ con la menor distancia tentativa (*delmin*) y se agrega a S.
2. Para cada arista (u, v) con $v \notin S$, si $d(u) + \ell(u, v) < d(v)$, se actualiza $d(v) \leftarrow d(u) + \ell(u, v)$ y el padre de v pasa a ser u (*decrease key*).

El proceso termina cuando ya no quedan nodos alcanzables fuera de S. Las aristas $(\text{padre}(v), v)$ forman el árbol de caminos más cortos.

3.2. Implementación

La cola de prioridad es un montículo binario (módulo `heapq`). Como `heapq` no permite disminuir la prioridad de un elemento, el *decrease key* se implementa insertando una nueva entrada con la distancia mejorada. Las entradas viejas de un nodo quedan en el montículo y se descartan al extraerlas si el nodo ya está en S. Cada entrada lleva además un contador de desempate, para que el montículo nunca tenga que comparar dos identificadores de nodo.

```python
def Dijkstra(self, s):
    self._validar_fuente(s)
    for arista in self.aristas:
        if arista.peso is not None and arista.peso < 0:
            raise ValueError("Dijkstra no admite aristas con peso negativo")

    distancia = {s: 0}
    padre = {s: None}
    explorados = []          # nodos en el orden en que entran a S
    en_s = set()
    desempate = itertools.count()   # evita comparar identificadores de nodos
    cola = [(0, next(desempate), s)]
    while cola:
        d_u, _, u = heapq.heappop(cola)          # delmin
        if u in en_s:
            continue                             # entrada obsoleta
        en_s.add(u)
        explorados.append(u)
        for v in self._vecinos[u]:
            if v in en_s:
                continue
            peso = self._pesos[(u, v)]
            nueva = d_u + (1 if peso is None else peso)
            if v not in distancia or nueva < distancia[v]:
                distancia[v] = nueva             # decrease key
                padre[v] = u
                heapq.heappush(cola, (nueva, next(desempate), v))

    arbol = Grafo(self.dirigido)
    etiqueta = {v: f"{v} ({distancia[v]:.2f})" for v in explorados}
    for v in explorados:
        arbol.agregar_nodo(etiqueta[v])
        if padre[v] is not None:
            arbol.agregar_arista(etiqueta[padre[v]], etiqueta[v], self._pesos[(padre[v],
    v)])
    return arbol
```

Código 1: Método `Dijkstra`.

El árbol resultante es un nuevo `Grafo`. Cada nodo se llama `"nodo (distancia)"`, con la distancia redondeada a dos decimales, y cada arista conserva su peso. El árbol contiene solo los nodos alcanzables desde s. Si hay pesos negativos el método lanza `ValueError`, porque Dijkstra no es correcto en ese caso, y si s no existe lanza `KeyError`. Las aristas sin peso cuentan como peso 1, en cuyo caso las distancias coinciden con las de BFS.

3.3. Complejidad y correctitud

Cada arista puede provocar a lo más una inserción en el montículo, así que este recibe $O(m)$ entradas, y cada inserción o extracción cuesta $O(\log m) = O(\log n)$. El costo total es $O((n+m)\log n)$.

La correctitud se basa en el teorema visto en clase: cuando un nodo v entra a S, su distancia tentativa ya es la distancia mínima. Si existiera un camino P más corto, tendría una primera arista (x, y) que sale de S, y como los pesos son no negativos, $L(P) \geq d(x) + \ell(x, y) \geq d(y) \geq d(v)$, porque v se eligió con la menor distancia tentativa. Eso contradice que P sea más corto.

3.4. Ejemplo

Como prueba se usó el grafo dirigido del ejemplo de clase, con origen s. La tabla 1 muestra el orden en que los nodos entran a S y la etiqueta que reciben en el árbol; coincide con el resultado de las diapositivas.

Orden	Nodo en el árbol	Padre
1	`s (0.00)`	—
2	`2 (9.00)`	s
3	`6 (14.00)`	s
4	`7 (15.00)`	s
5	`3 (32.00)`	6
6	`5 (34.00)`	3
7	`4 (45.00)`	5
8	`t (50.00)`	5

Tabla 1: Resultado de `Dijkstra("s")` en el ejemplo de clase.

3.5. Verificación

Además del ejemplo anterior, se comprobó con un programa de prueba que, en grafos de los cinco modelos aleatorios con 30 y 500 nodos y pesos aleatorios, se cumple lo siguiente:

- La distancia de cada nodo coincide con la calculada de forma independiente por el algoritmo de Bellman–Ford.
- Cada etiqueta tiene el formato `"nodo (distancia)"`, con dos decimales.
- El árbol tiene un nodo por cada nodo alcanzable, un solo padre por nodo, $n - 1$ aristas, y cada arista conserva el peso del grafo original.
- Para cada arista (u, v) del árbol, $d(v) = d(u) + \ell(u, v)$; es decir, todas las aristas del árbol están en caminos más cortos.
- El método también funciona en grafos dirigidos, sin pesos o no conexos, y rechaza pesos negativos y nodos fuente inexistentes.

Todo el código, incluidos los scripts de generación y de imágenes, se ejecutó con Python 3.6.8, la versión solicitada.

4. Experimentos

El script `generar_gv.py` genera dos grafos por cada uno de los cinco generadores aleatorios: uno con “pocos” nodos (30) y otro con “muchos” (500). La malla no se usa porque no es aleatoria. Cada grafo se vuelve a generar hasta que resulta conexo, de modo que el árbol abarca todos sus nodos. Después se le asignan pesos aleatorios uniformes entre 1 y 100 y se calcula su árbol de caminos más cortos desde el nodo 0.

Modelo	Parámetros
Erdős–Rényi	$m = \operatorname{round}\left(\dfrac{n(\ln n + 2)}{2}\right)$
Gilbert	$p = \dfrac{\ln n + 3}{n}$
Geográfico simple	$r = \sqrt{\dfrac{\ln n + 4}{\pi n}}$
Barabási–Albert	$d = 4$
Dorogovtsev–Mendes	sin parámetros adicionales

Tabla 2: Parámetros de los generadores.

Los grafos se guardan en `gv/generados/` y los árboles en `gv/calculados/`. El script `generar_imagenes.py` los dibuja con Graphviz. Los grafos generados se dibujan con `sfdp`, y en los de 30 nodos se escribe el peso sobre cada arista. Los árboles se dibujan con `twopi`, con la raíz al centro y la etiqueta `nodo (distancia)` en cada nodo.

5. Resultados

La tabla 3 resume los árboles obtenidos. La *altura* es el mayor número de aristas en un camino del árbol desde s. Como comparación se incluye la altura del árbol BFS del mismo grafo, que ignora los pesos. Los grafos y los pesos son aleatorios, así que los valores corresponden a la ejecución incluida en el repositorio.

			Distancia a s		Altura	
Grafo	Nodos	Aristas	Máxima	Promedio	Dijkstra	BFS
Erdős–Rényi	30	81	122.92	63.63	6	3
Gilbert	30	83	117.53	55.20	4	3
Geográfico simple	30	84	238.10	61.61	8	8
Barabási–Albert	30	56	194.53	105.90	9	6
Dorogovtsev–Mendes	30	57	188.03	102.24	5	4
Erdős–Rényi	500	2054	162.04	88.63	10	5
Gilbert	500	2281	125.69	68.82	10	4
Geográfico simple	500	2361	341.20	149.57	26	17
Barabási–Albert	500	997	3390.00	1747.04	114	89
Dorogovtsev–Mendes	500	997	300.25	116.36	9	6

Tabla 3: Distancias y alturas de los árboles de caminos más cortos.

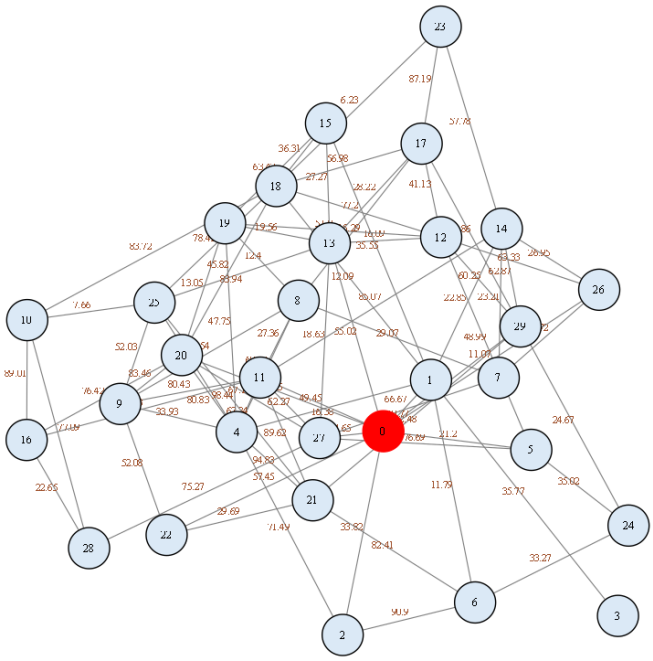
De la tabla se desprenden varias observaciones:

- **Dijkstra produce árboles más altos que BFS.** En casi todos los grafos, el camino más corto por peso usa más aristas que el camino con menos aristas: conviene dar varios pasos por aristas ligeras antes que uno solo por una arista pesada. Con 500 nodos, el árbol de Erdős–Rényi tiene altura 10 con Dijkstra y 5 con BFS.

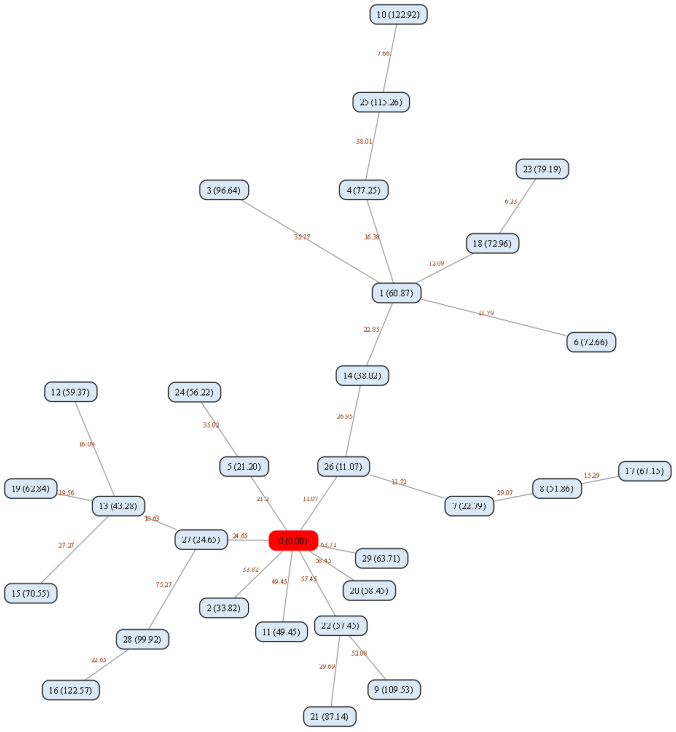
- **En los grafos densos las distancias apenas crecen.** En Erdős–Rényi y Gilbert, la distancia máxima pasa de unos 120 con 30 nodos a entre 125 y 162 con 500 nodos. Con más aristas hay más rutas alternativas y es más fácil encontrar una formada por aristas ligeras.
- **Barabási–Albert tiene las distancias más grandes.** La variante implementada limita el grado de cada nodo a $d = 4$, lo que produce caminos muy largos: con 500 nodos, el nodo más lejano está a 114 aristas y a una distancia de 3390.
- **En el modelo geográfico la altura crece con el tamaño.** Como las aristas solo unen nodos cercanos, llegar a los nodos lejanos requiere muchas aristas, igual que en BFS.

6. Visualización

Las figuras 1 a 10 muestran cada grafo generado junto con su árbol de caminos más cortos. En los árboles de 500 nodos las etiquetas son pequeñas; se leen con zoom en el PDF o en las imágenes de `imagenes/calculados/`.

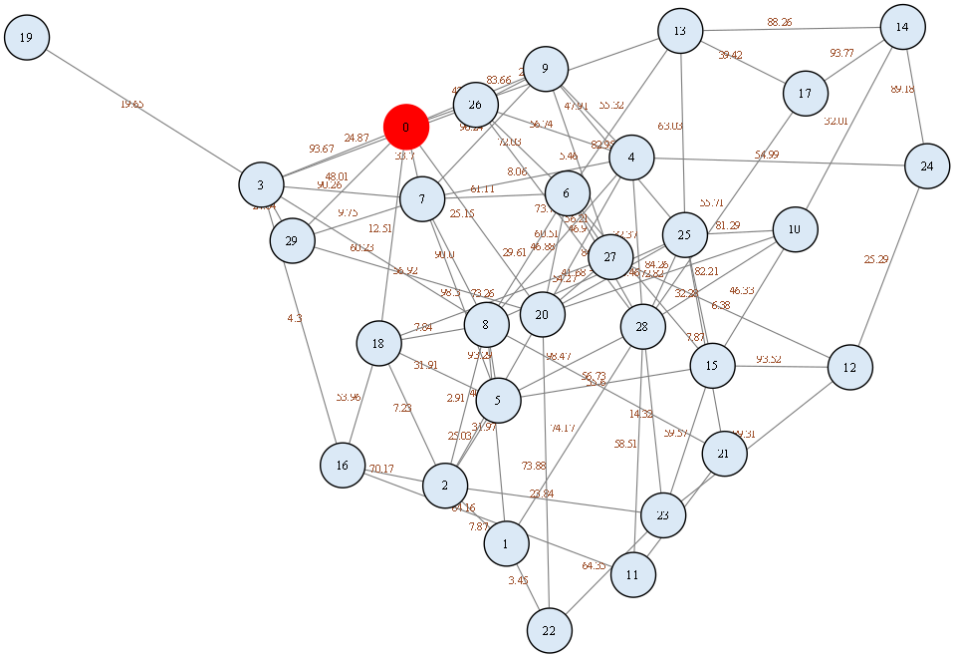


(a) Grafo generado

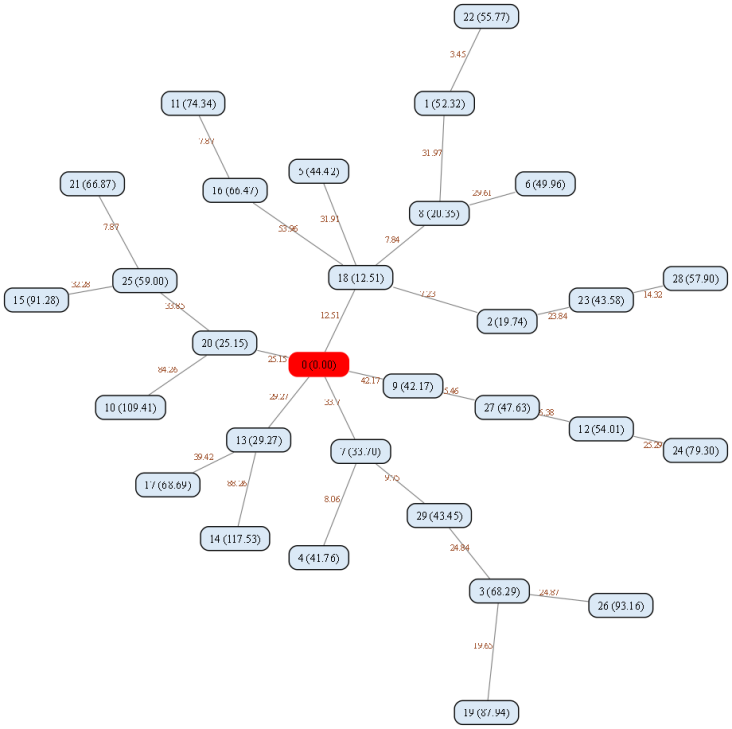


(b) Árbol de caminos más cortos

Figura 1: Erdős–Rényi con 30 nodos. El nodo fuente aparece en rojo.

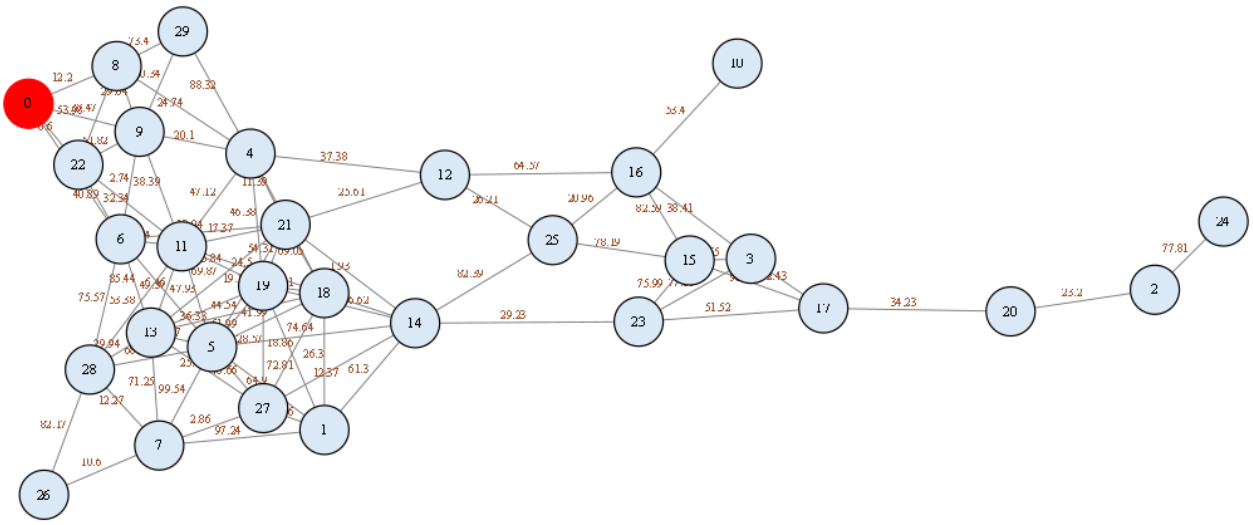


(a) Grafo generado

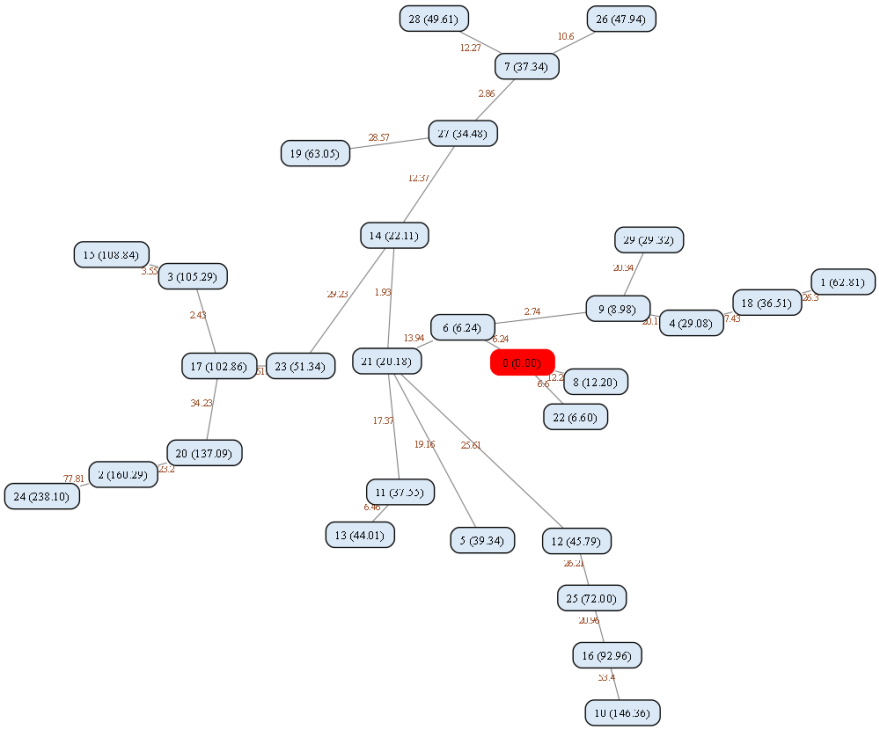


(b) Árbol de caminos más cortos

Figura 2: Gilbert con 30 nodos. El nodo fuente aparece en rojo.

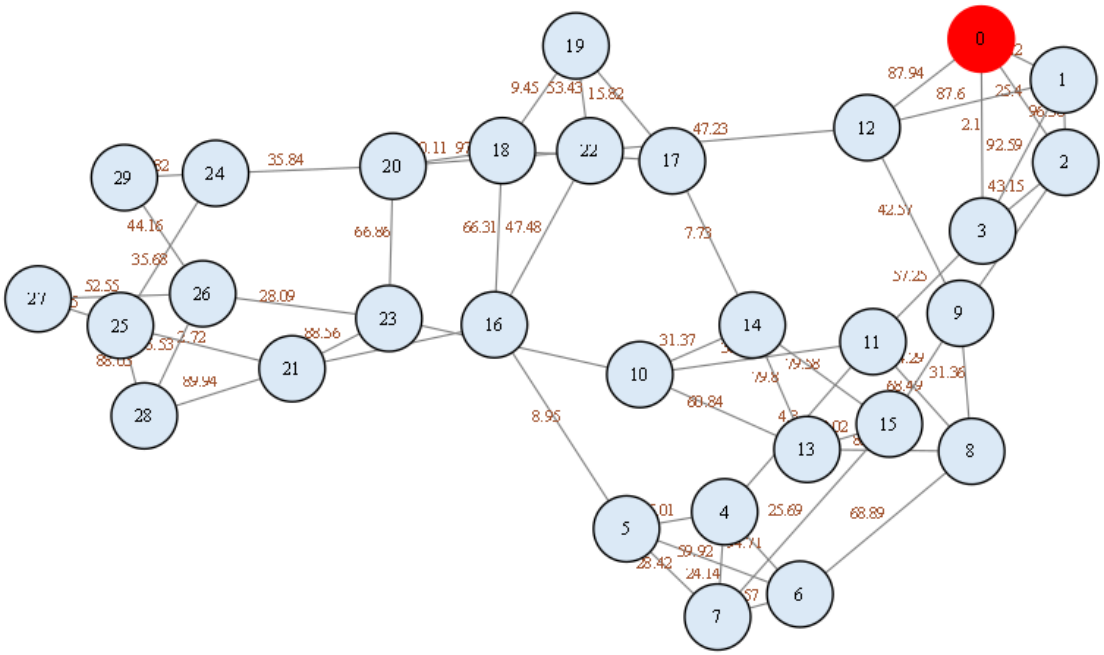


(a) Grafo generado

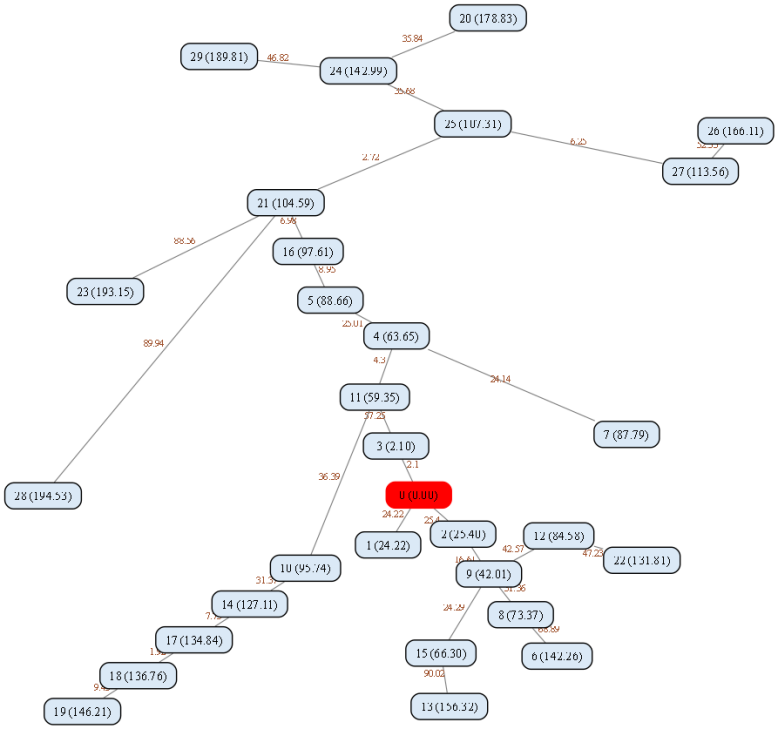


(b) Árbol de caminos más cortos

Figura 3: Geográfico simple con 30 nodos. El nodo fuente aparece en rojo.

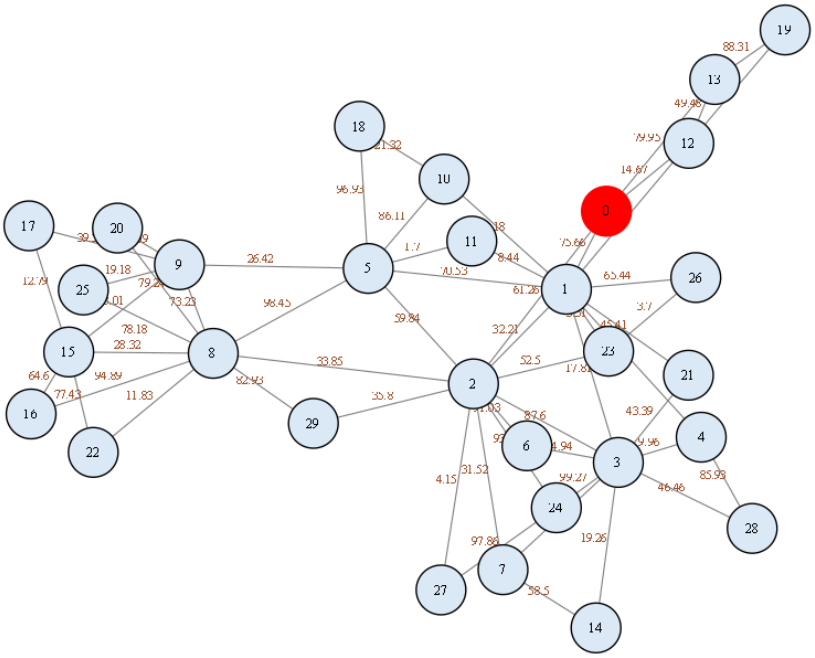


(a) Grafo generado

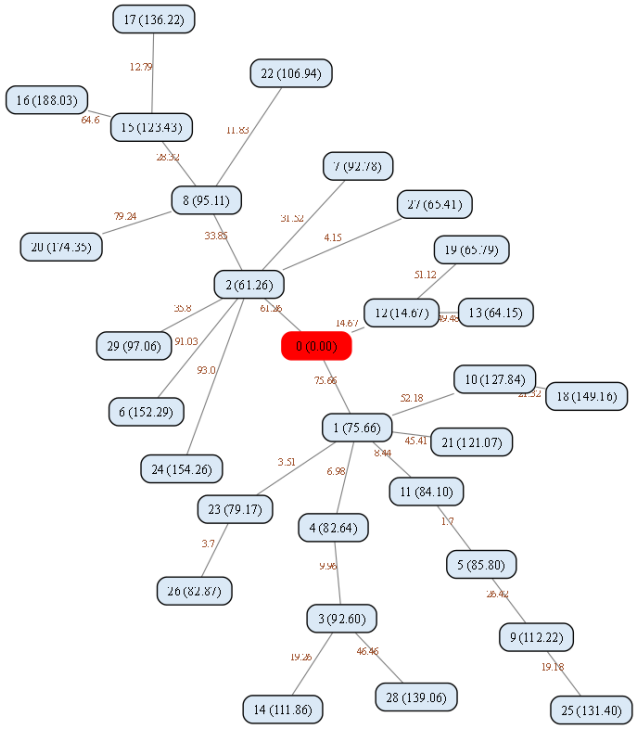


(b) Árbol de caminos más cortos

Figura 4: Barabási–Albert con 30 nodos. El nodo fuente aparece en rojo.

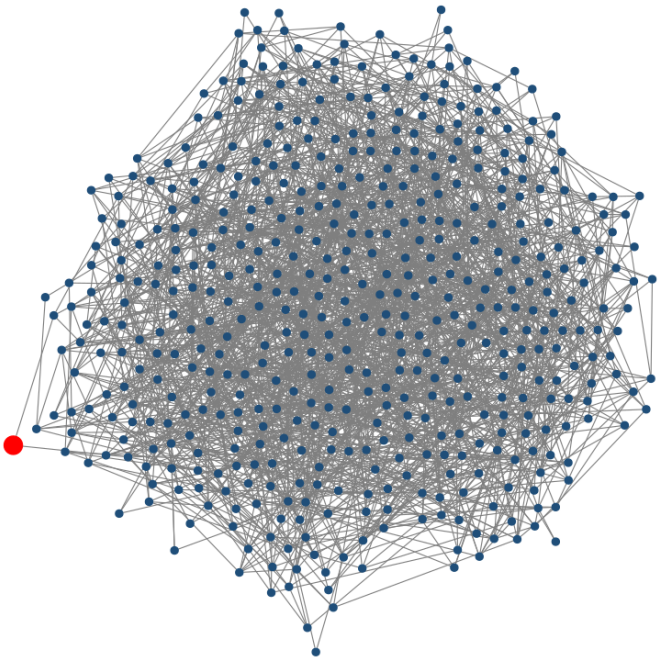


(a) Grafo generado



(b) Árbol de caminos más cortos

Figura 5: Dorogovtsev–Mendes con 30 nodos. El nodo fuente aparece en rojo.

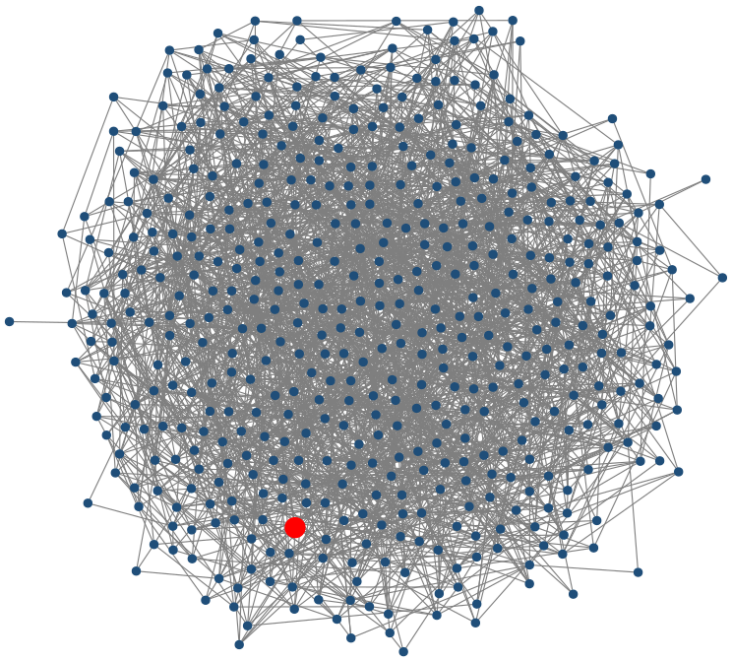


(a) Grafo generado

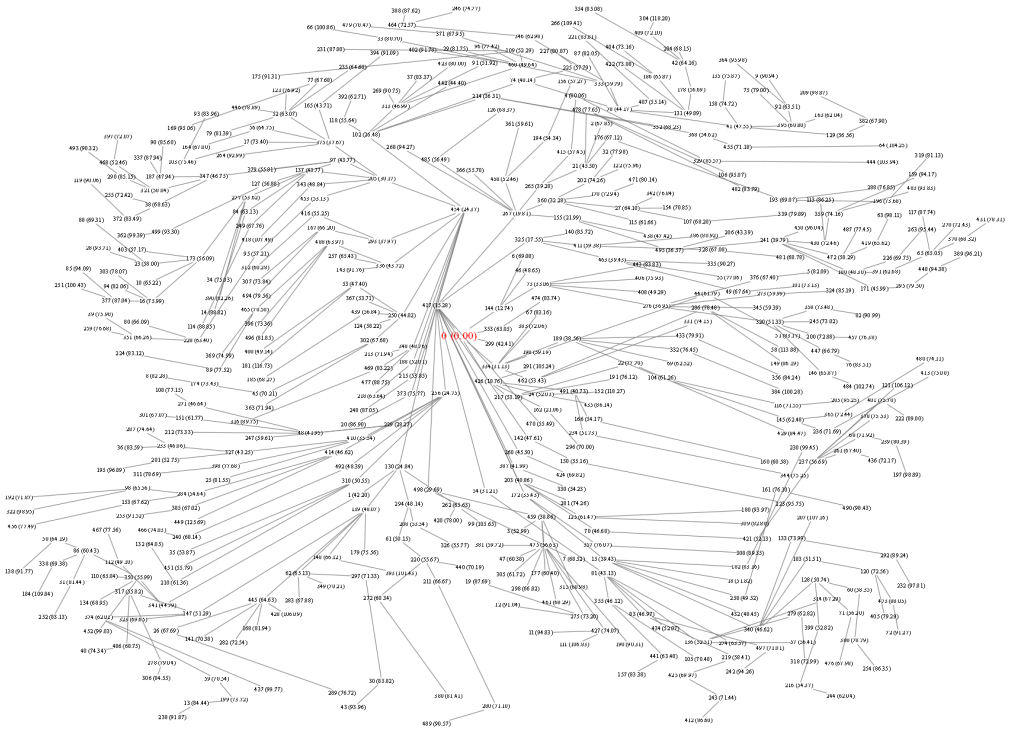


(b) Árbol de caminos más cortos

Figura 6: Erdős–Rényi con 500 nodos. El nodo fuente aparece en rojo.

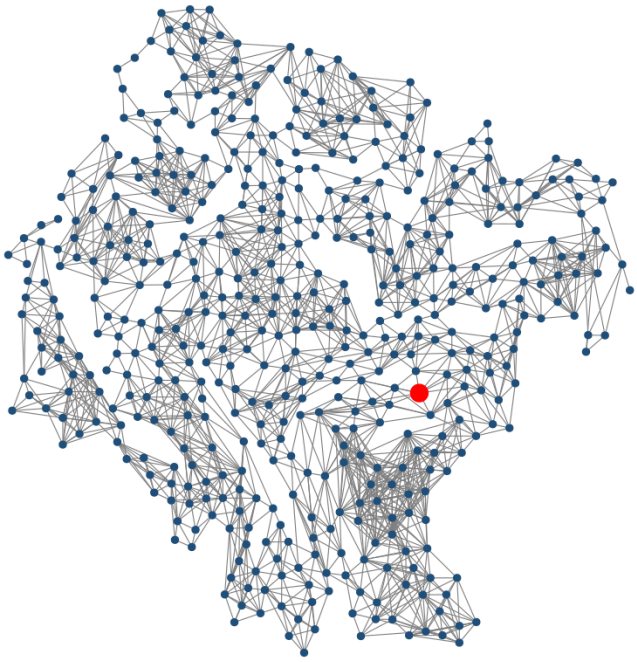


(a) Grafo generado

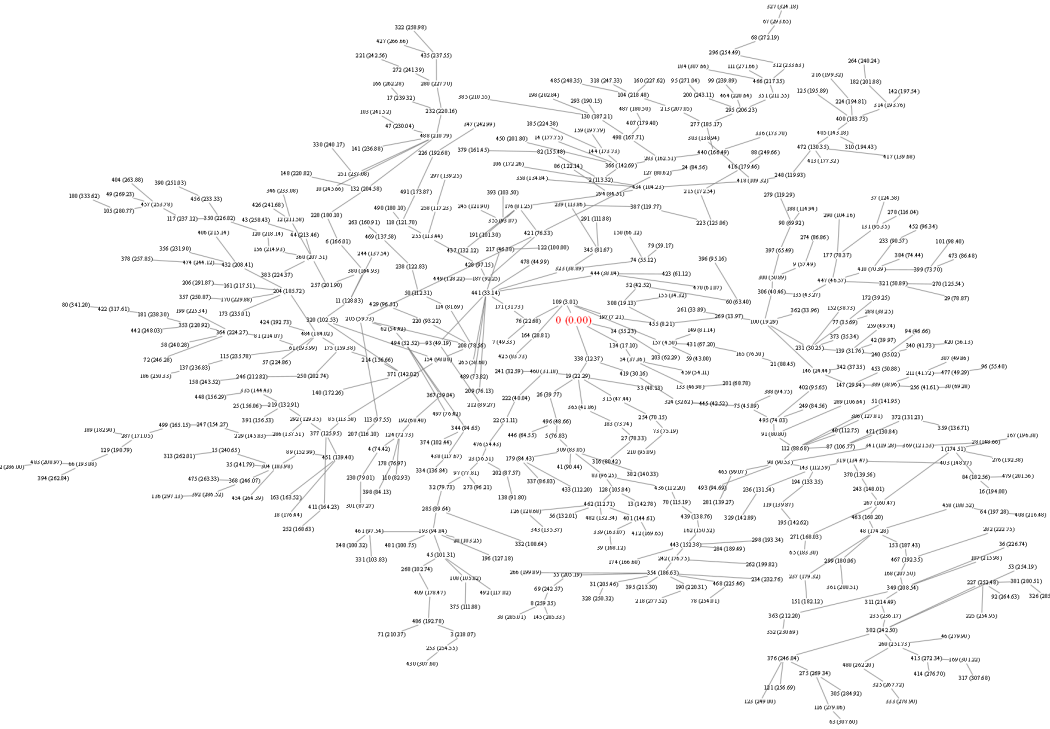


(b) Árbol de caminos más cortos

Figura 7: Gilbert con 500 nodos. El nodo fuente aparece en rojo.

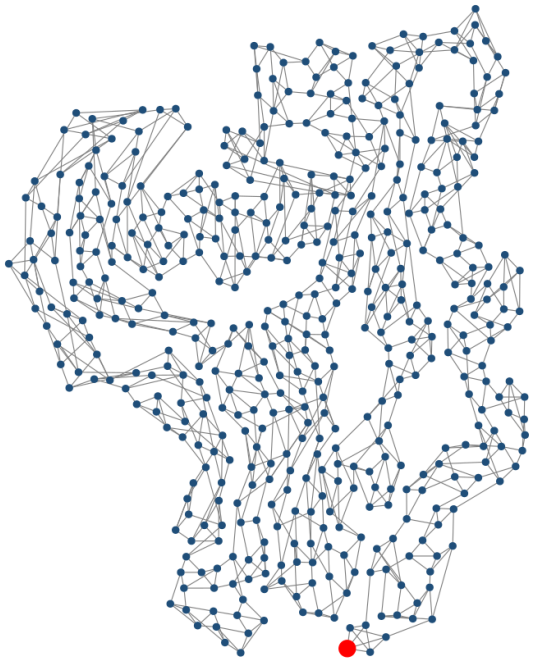


(a) Grafo generado

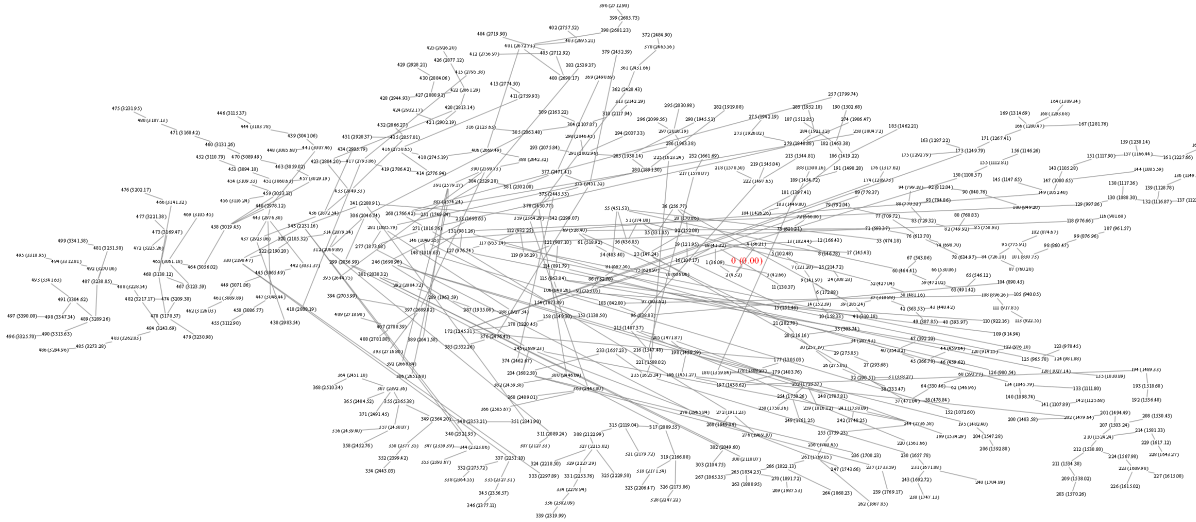


(b) Árbol de caminos más cortos

Figura 8: Geográfico simple con 500 nodos. El nodo fuente aparece en rojo.

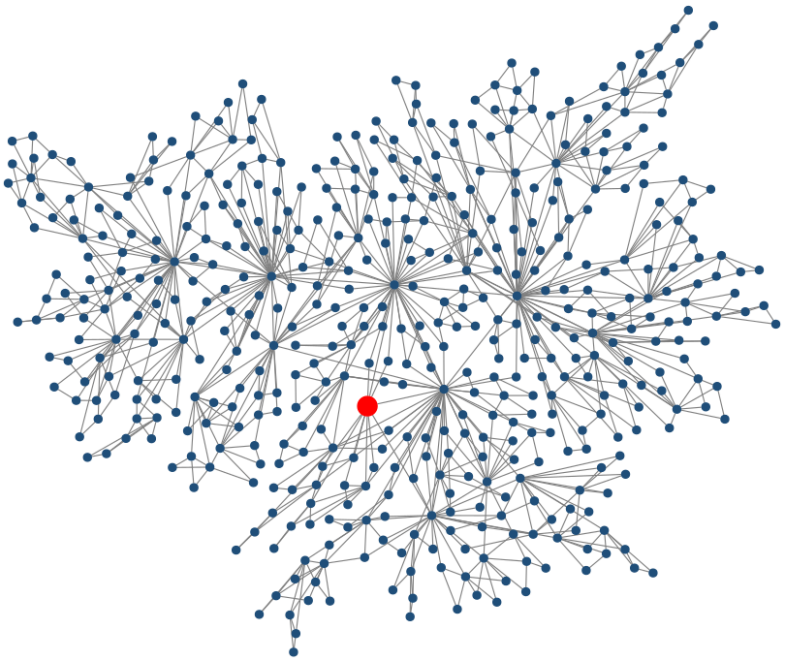


(a) Grafo generado

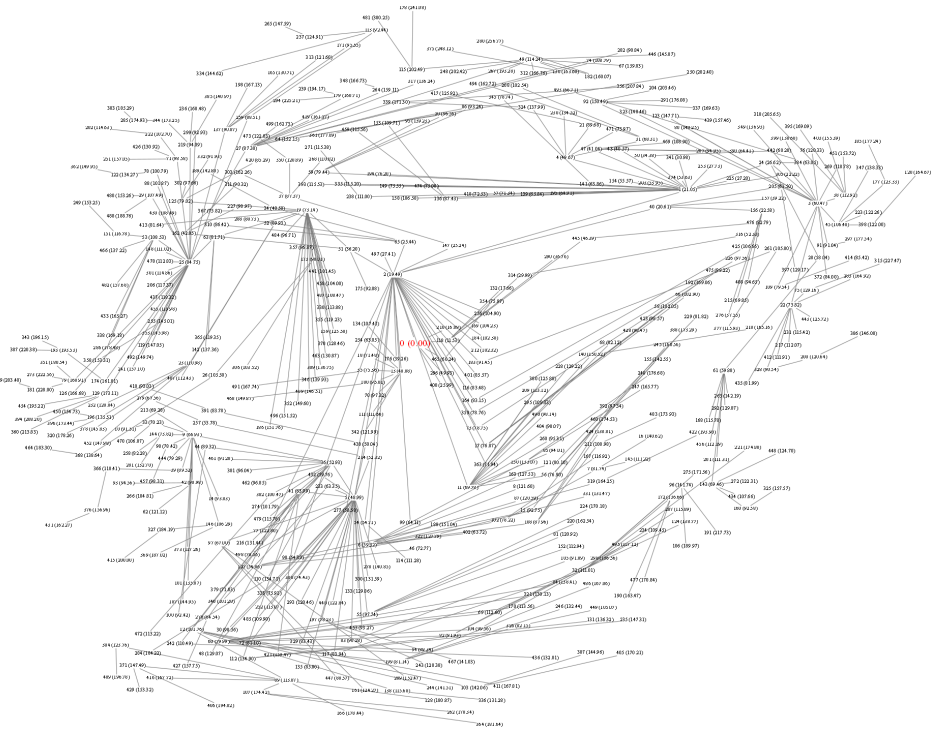


(b) Árbol de caminos más cortos

Figura 9: Barabási–Albert con 500 nodos. El nodo fuente aparece en rojo.



(a) Grafo generado



(b) Árbol de caminos más cortos

Figura 10: Dorogovtsev–Mendes con 500 nodos. El nodo fuente aparece en rojo.